AT-10-20 — Calculadora de Bhaskara com JavaFX e MVC

1. Aplicação desenvolvida

Escolhi fazer uma calculadora de Bhaskara. Assim como a calculadora de IMC do tutorial, ela recebe valores digitados pelo usuário, faz um cálculo e mostra o resultado em uma janela. A diferença é que, neste caso, a entrada são os coeficientes **a**, **b** e **c** de uma equação do segundo grau.

A equação tem a forma $\mathbf{ax^2 + bx + c = 0}$, com $\mathbf{a \neq 0}$. O programa calcula primeiro o discriminante, $\mathbf{\Delta = b^2 - 4ac}$, e depois usa as fórmulas:

$$\mathbf{x_1 = (-b - \sqrt{\Delta}) / (2a)}$$

$$\mathbf{x_2 = (-b + \sqrt{\Delta}) / (2a)}$$

Se delta for positivo, aparecem duas raízes. Se for zero, aparece uma raiz dupla. Se for negativo, a tela informa que não existem raízes reais. Também coloquei um botão para limpar os campos e mensagens para entradas inválidas, como letras no lugar dos números ou o coeficiente a igual a zero.

2. Organização do código

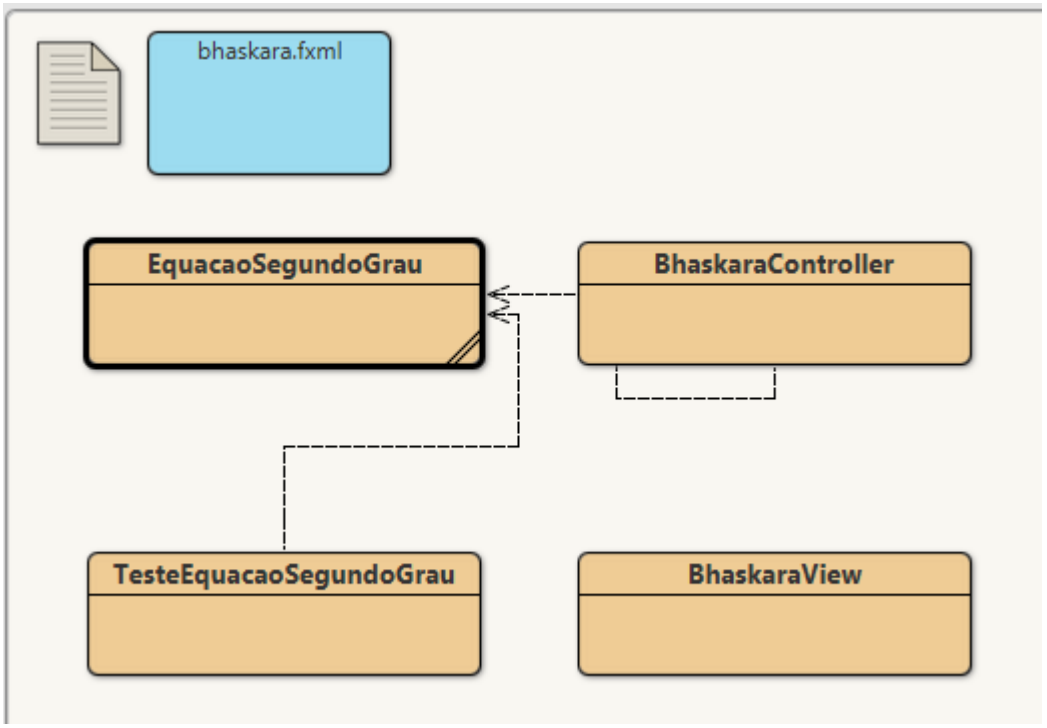
Para seguir o padrão MVC, dividi o programa em três partes:

Model — EquacaoSegundoGrau.java: guarda os coeficientes e faz os cálculos. O método calcularDelta() encontra o discriminante e calcularRaizesReais() devolve as raízes. Essa classe não precisa do JavaFX.

View — BhaskaraView.java e bhaskara.fxml: o arquivo FXML organiza a tela, com três caixas de texto, os botões Calcular e Limpar e os espaços onde aparecem os resultados. A classe BhaskaraView carrega esse arquivo e abre a janela.

Controller — BhaskaraController.java: recebe os cliques dos botões. Ao clicar em Calcular, o controlador lê os campos, cria um objeto de EquacaoSegundoGrau e mostra na tela o resultado recebido do Model. Ao clicar em Limpar, ele apaga os dados e os resultados.

No FXML, usei fx:id para identificar os campos e onAction para ligar os botões aos métodos do controlador.

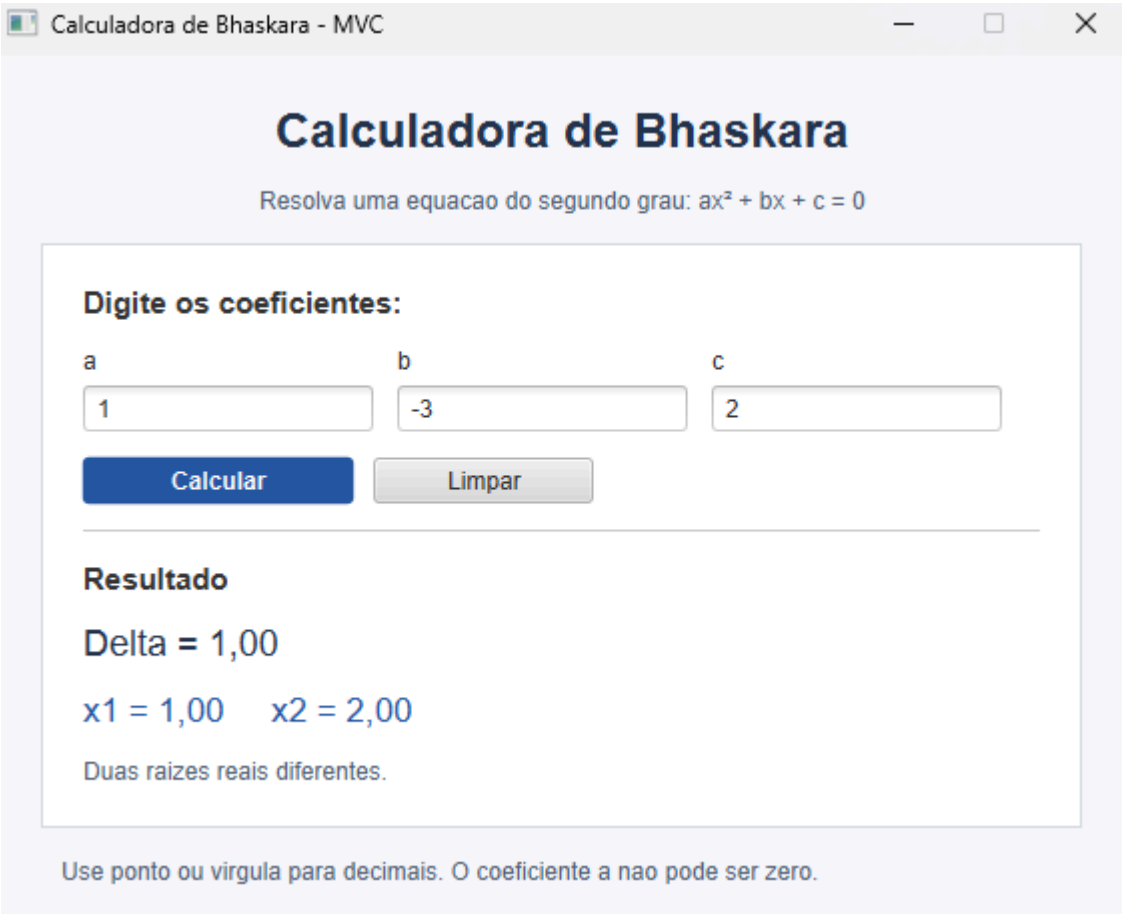


3. Testes da calculadora

Separei alguns valores para conferir os resultados e verificar se a interface responde corretamente às entradas.

Primeiro teste: duas raízes reais

Usei **a = 1**, **b = −3** e **c = 2**. A equação é **$x^2 - 3x + 2 = 0$**. Nesse caso, o delta é **1**, e as raízes esperadas são **1** e **2**.



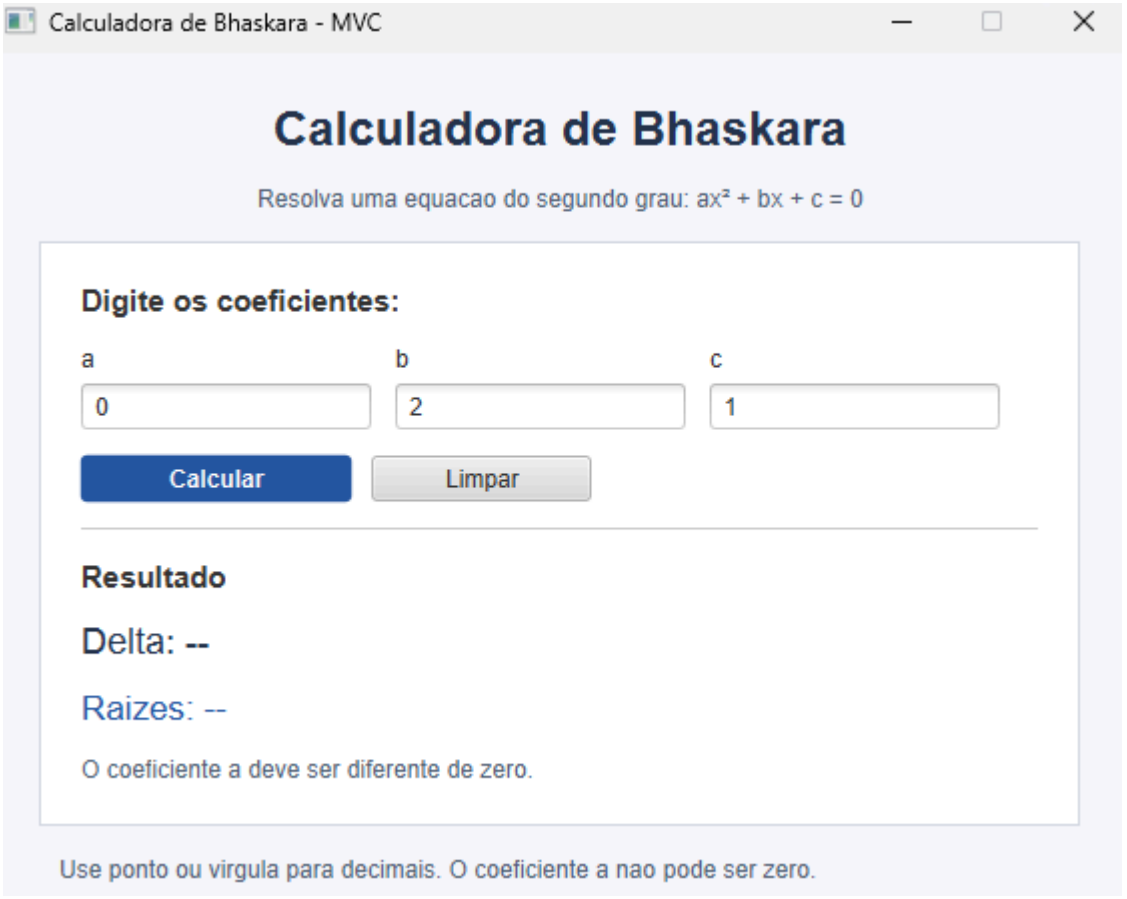
Segundo teste: sem raízes reais

Para conferir outro caso, os valores escolhidos foram **a = 1**, **b = 0** e **c = 1**. O delta dessa equação é **−4**, então não há raízes reais.



Terceiro teste: entrada inválida

Também preparei um teste com **a = 0**, **b = 2** e **c = 1**. Como a precisa ser diferente de zero, o programa não deve tentar calcular as raízes e deve mostrar uma mensagem de erro.


Calculadora de Bhaskara - MVC
Calculadora de Bhaskara
Resolva uma equacao do segundo grau: ax² + bx + c = 0
Digite os coeficientes:
a
b
c
0
2
1
Calcular
Limpar
Resultado
Delta: --
Raizes: --
O coeficiente a deve ser diferente de zero.
Use ponto ou virgula para decimais. O coeficiente a nao pode ser zero.

Códigos:

EquacaoSegundoGrau

Compilar | Desfazer | Recortar | Copiar | Colar | Procurar... | Fechar

```java
/**
 * Model: guarda os coeficientes e realiza os calculos de Bhaskara.
 * Esta classe nao depende do JavaFX.
 */
public class EquacaoSegundoGrau
{
    private double a;
    private double b;
    private double c;

    public EquacaoSegundoGrau(double a, double b, double c)
    {
        if (!Double.isFinite(a) || !Double.isFinite(b) || !Double.isFinite(c)) {
            throw new IllegalArgumentException("Informe numeros validos.");
        }
        if (a == 0) {
            throw new IllegalArgumentException("O coeficiente a deve ser diferente de zero.");
        }

        this.a = a;
        this.b = b;
        this.c = c;
    }

    /** Calcula o discriminante: delta = b*b - 4*a*c. */
    public double calcularDelta()
    {
        return b * b - 4 * a * c;
    }

    /**
     * Devolve as raizes reais em um vetor:
     * 0 posicoes: nenhuma raiz real;
     * 1 posicao: uma raiz real dupla;
     * 2 posicoes: duas raizes reais diferentes.
     */
    public double[] calcularRaizesReais()
    {
        double delta = calcularDelta();

        if (!Double.isFinite(delta)) {
            throw new IllegalArgumentException("Valores muito grandes para o calculo.");
        }
        if (delta < 0) {
            return new double[0];
        }
        if (delta == 0) {
            return new double[] { -b / (2 * a) };
        }

        double raizDelta = Math.sqrt(delta);
        double x1 = (-b - raizDelta) / (2 * a);
        double x2 = (-b + raizDelta) / (2 * a);
        return new double[] { x1, x2 };
    }
}
```

1.

EquacaoSegundoGrau | BhaskaraController | TesteEquacaoSegundoGrau | BhaskaraView

Compilar | Desfazer | Recortar | Copiar | Colar | Procurar... | Fechar

2.

```java
import java.util.Locale;
import javafx.fxml.FXML;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;

/**
 * Controller: le os campos, chama o Model e mostra os resultados.
 */
public class BhaskaraController
{
    @FXML private TextField campoA;
    @FXML private TextField campoB;
    @FXML private TextField campoC;
    @FXML private Label labelDelta;
    @FXML private Label labelRaizes;
    @FXML private Label labelMensagem;

    @FXML
    private void botaoCalcularClicado()
    {
        limparResultados();

        try {
            double a = lerNumero(campoA);
            double b = lerNumero(campoB);
            double c = lerNumero(campoC);

            EquacaoSegundoGrau equacao = new EquacaoSegundoGrau(a, b, c);
            double delta = equacao.calcularDelta();
            double[] raizes = equacao.calcularRaizesReais();

            labelDelta.setText("Delta = " + formatar(delta));

            if (raizes.length == 0) {
                labelRaizes.setText("Nao existem raizes reais.");
                labelMensagem.setText("O delta e negativo.");
            } else if (raizes.length == 1) {
                labelRaizes.setText("x = " + formatar(raizes[0]));
                labelMensagem.setText("Uma raiz real dupla.");
            } else {
                labelRaizes.setText("x1 = " + formatar(raizes[0])
                        + "     x2 = " + formatar(raizes[1]));
                labelMensagem.setText("Duas raizes reais diferentes.");
            }
        } catch (NumberFormatException erro) {
            labelMensagem.setText("Preencha os tres campos com numeros.");
        } catch (IllegalArgumentException erro) {
            labelMensagem.setText(erro.getMessage());
        }
    }

    @FXML
    private void botaoLimparClicado()
    {
        campoA.clear();
        campoB.clear();
        campoC.clear();
        limparResultados();
        campoA.requestFocus();
    }
```

```
62      private void limparResultados()
63      {
64          labelDelta.setText("Delta: --");
65          labelRaizes.setText("Raizes: --");
66          labelMensagem.setText("Informe os coeficientes e clique em Calcular.");
67      }
68
69      private double lerNumero(TextField campo)
70      {
71          return Double.parseDouble(campo.getText().trim().replace(',', '.'));
72      }
73
74      private String formatar(double numero)
75      {
76          return String.format(Locale.forLanguageTag("pt-BR"), "%.2f", numero);
77      }
78  }
```

EquacaoSegundoGrau | BhaskaraController | TesteEquacaoSegundoGrau | BhaskaraView

Compilar | Desfazer | Recortar | Copiar | Colar | Procurar... | Fechar

```java
/**
 * Pequenos testes do Model. Nao e necessario abrir a janela do JavaFX.
 * No BlueJ, execute o metodo main desta classe.
 */
public class TesteEquacaoSegundoGrau
{
    private static int total = 0;

    public static void main(String[] args)
    {
        testar(1, -3, 2, 1, new double[] {1, 2});
        testar(1, 2, 1, 0, new double[] {-1});
        testar(1, 0, 1, -4, new double[] {});
        testar(2, 0, -8, 64, new double[] {-2, 2});
        testar(1.5, -3, 1.5, 0, new double[] {1});
        testar(-1, 3, -2, 1, new double[] {2, 1});

        testarCoeficienteAZero();
        testarNumeroInvalido();
        testarValoresMuitoGrandes();
        System.out.println("OK: " + total + " testes passaram.");
    }

    private static void testar(double a, double b, double c,
                               double deltaEsperado, double[] esperado)
    {
        EquacaoSegundoGrau e = new EquacaoSegundoGrau(a, b, c);
        conferir(e.calcularDelta(), deltaEsperado);

        double[] obtido = e.calcularRaizesReais();
        if (obtido.length != esperado.length) {
            throw new AssertionError("Quantidade de raizes diferente do esperado.");
        }
        for (int i = 0; i < obtido.length; i++) {
            conferir(obtido[i], esperado[i]);
        }
        total++;
    }

    private static void conferir(double obtido, double esperado)
    {
        if (Math.abs(obtido - esperado) > 0.000001) {
            throw new AssertionError("Esperado " + esperado + ", mas foi " + obtido);
        }
    }

    private static void testarCoeficienteAZero()
    {
        try {
            new EquacaoSegundoGrau(0, 2, 1);
            throw new AssertionError("Deveria rejeitar a = 0.");
        } catch (IllegalArgumentException erro) {
            total++;
        }
    }
```

3.

```
56
57      private static void testarNumeroInvalido()
58      {
59          try {
60              new EquacaoSegundoGrau(Double.NaN, 2, 1);
61              throw new AssertionError("Deveria rejeitar NaN.");
62          } catch (IllegalArgumentException erro) {
63              total++;
64          }
65      }
66
67      private static void testarValoresMuitoGrandes()
68      {
69          try {
70              new EquacaoSegundoGrau(1e200, 1e200, -1e200).calcularRaizesReais();
71              throw new AssertionError("Deveria rejeitar delta infinito.");
72          } catch (IllegalArgumentException erro) {
73              total++;
74          }
75      }
76  }
77
```

EquacaoSegundoGrau | BhaskaraController | TesteEquacaoSegundoGrau | BhaskaraView

Compilar | Desfazer | Recortar | Copiar | Colar | Procurar... | Fechar

4.

```
1   import java.io.File;
2   import javafx.application.Application;
3   import javafx.fxml.FXMLLoader;
4   import javafx.scene.Parent;
5   import javafx.scene.Scene;
6   import javafx.stage.Stage;
7
8   /**
9    * View: abre a janela definida no arquivo bhaskara.fxml.
10   * Nao possui as formulas da equacao.
11   */
12  public class BhaskaraView extends Application
13  {
14      @Override
15      public void start(Stage palco) throws Exception
16      {
17          File arquivo = new File("bhaskara.fxml");
18          Parent painel = FXMLLoader.load(arquivo.toURI().toURL());
19
20          palco.setTitle("Calculadora de Bhaskara - MVC");
21          palco.setScene(new Scene(painel));
22          palco.setResizable(false);
23          palco.show();
24      }
25  }
26
```